

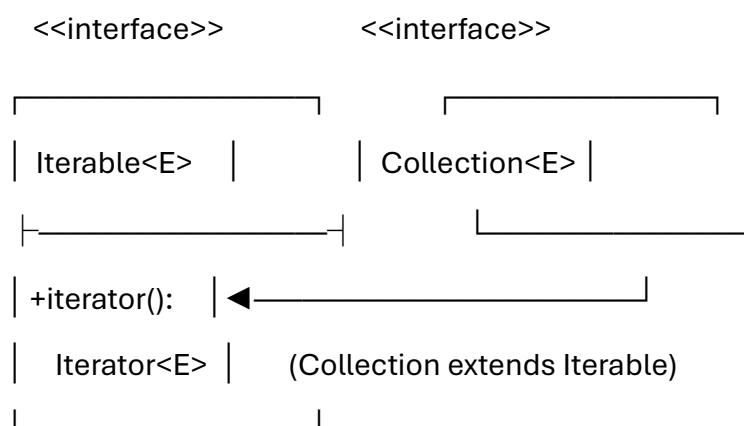
Aim

To write a Java program that demonstrates the Iterator interface methods hasNext(), next(), remove() and forEachRemaining() by traversing an ArrayList and removing elements safely during traversal.

Algorithm

1. Start.
2. Import the java.util package.
3. Create a List<String> object using ArrayList and add the elements Apple, Banana, Cherry, Date and Elderberry. Display the list.
4. Obtain an Iterator from the list using iterator().
5. Traverse the list:
 - Repeat while hasNext() returns true.
 - In each iteration, call next() to get the next element and display it.
6. Display the value of hasNext() after the loop. It is false.
7. Obtain a new iterator and traverse the list again. When the element is "Banana", call remove() to delete it from the underlying list.
8. Display the list after the removal.
9. Obtain another iterator, call next() once to read the first element, then call forEachRemaining() with a lambda to process all the remaining elements.
10. Obtain a new iterator and call remove() before calling next(). Catch the IllegalStateException and display it.
11. Stop.

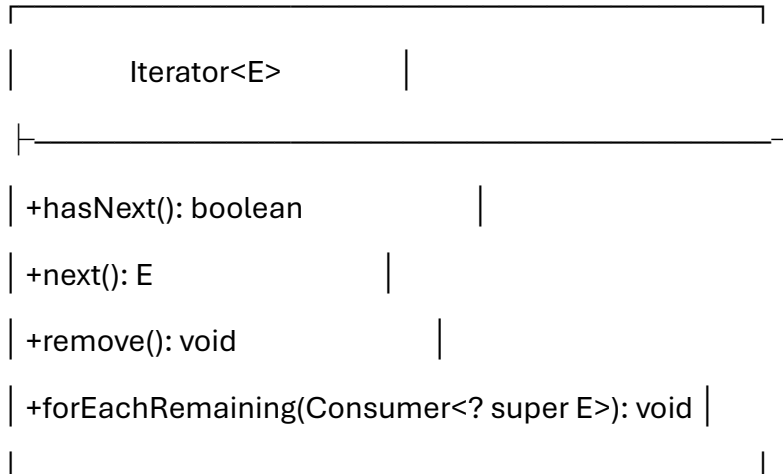
UML Diagram



| creates



<<interface>>



| (implements)



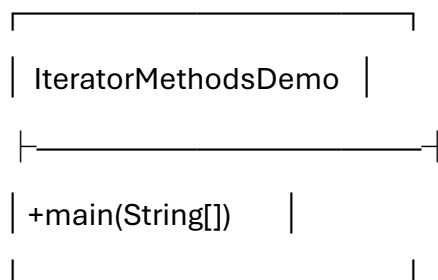
| (extends)

ArrayList's

<<interface>>

internal iterator

ListIterator<E>



(uses Iterator<String>)

Program:

```
package Observations;
```

```
import java.util.*;
```

```
public class IteratorInterface {
```

```
    public static void main(String[] args) {
```

```

List<String> list = new ArrayList<>(
    Arrays.asList("Apple", "Banana", "Cherry", "Date", "Elderberry"));
System.out.println("Original list      : " + list);

// hasNext() and next()
Iterator<String> it = list.iterator();
System.out.println("Traversal using hasNext() and next():");
while (it.hasNext()) {
    System.out.println(it.next());
}
System.out.println("hasNext() after loop : " + it.hasNext());

// remove()
Iterator<String> it2 = list.iterator();
while (it2.hasNext()) {
    String fruit = it2.next();
    if (fruit.equals("Banana")) {
        it2.remove();
    }
}
System.out.println("List after remove()  : " + list);

// forEachRemaining()
Iterator<String> it3 = list.iterator();
System.out.println("First element via next(): " + it3.next());
System.out.println("forEachRemaining():");
it3.forEachRemaining(fruit -> System.out.println(fruit));

```

```

// remove() before next() is illegal

Iterator<String> it4 = list.iterator();

try {
    it4.remove();
} catch (IllegalStateException e) {
    System.out.println("remove() before next() -> Exception: " + e);
}
}
}

```

Output:

```

es' '-cp' 'C:\Users\Sahithi\pyla\AppData\Roaming\Code\User\workspaceStorage\234db
bservations.IteratorInterface'
Original list      : [Apple, Banana, Cherry, Date, Elderberry]
Traversal using hasNext() and next():
Apple
Banana
Cherry
Date
Elderberry
hasNext() after loop : false
List after remove()  : [Apple, Cherry, Date, Elderberry]
First element via next(): Apple
forEachRemaining():
Cherry
Date
Elderberry
remove() before next() -> Exception: java.lang.IllegalStateException
PS C:\Users\Sahithi\pyla\Desktop\Sahithi_java>

```

Result:

Thus, the Java program demonstrating the Iterator interface methods was executed successfully and the output was verified.

- hasNext() checks whether more elements remain, and next() returns the next element and moves forward.

- `remove()` deletes the last element returned by `next()` from the underlying collection. It can be called only once per `next()` call, and calling it before `next()` throws `IllegalStateException`.
- `forEachRemaining()` applies an action to all the elements that have not yet been visited, so it consumes the iterator.

`Iterator.remove()` is the safe way to delete elements during traversal. If you call `list.remove()` directly inside the loop instead, the next call to `next()` throws `ConcurrentModificationException`